

PaCHITA: A Patched Channel-Independent Transformer for Business Process Anomaly Detection

2026.08.25

Yongjae Lee¹, Mingyun Kang², and Hyerim Bae^{2*}

¹Hasso Plattner Institute, University of Potsdam, Potsdam, Germany

²Major in Industrial Data Science & Engineering, Department of Industrial Engineering
Pusan National University, Busan 46241, Republic of Korea

Contents

1. Introduction
2. Background
3. Proposed Method
4. Experiments
5. Conclusion

01. Introduction

➤ Business Process Management (BPM) and Process Mining

- **Process Mining in BPM**

- BPM aims to manage and improve chains of events, activities, and decisions that create organizational value [1].
- Process mining analyzes actual process behavior using event log recorded by information systems [2].

- **Event Logs in Process Mining**

- Process analysis relies on event logs collected from process-aware information systems [2].
- Each event log records process executions as traces of ordered events with associated attributes.

- **Anomalies in Event Logs**

- The reliability of process analysis depends strongly on the quality of the underlying event log [3].
- Nevertheless, anomalies in real-world event logs are inevitable [4].
- Detecting anomalies enables corrective actions before unreliable data affects process decisions.

Business Process Anomaly Detection (BPAD)
is crucial for reliable process analysis.

01. Introduction

➤ Business Process Anomaly Detection (BPAD)

• Existing BPAD Methods

- Early approaches relied on standard process models and conformance checking [5-10].
- Statistical methods used statistical deviation scores or stochastic models to identify unusual event behavior [11-15].

• Unsupervised Deep Learning Methods

- Recent approaches learn representations directly from event logs, eliminating the need for prior process knowledge [16-22].
- DL-based approaches detect anomalies through reconstruction, graph representations, or recurrent prediction [16-22].

Category	Related Work	Key Characteristic
Process mining	Naïve, Sampling [8]	Naïve: detects infrequent trace variants. Sampling: mines a process model from sampled traces and checks conformance.
Statistical methods	Statistical leverage [12]; Extended dynamic Bayesian networks [13]; Markov and hidden Markov models [14, 15]	Statistical deviation scores or stochastic models are used to identify unusual event behavior.
Deep learning	VAE, LAE [17]; GAE [18]; BINet [19]; GRASPED [20]	Anomalies are detected through reconstruction, graph representations, or recurrent prediction of activity and attribute behavior.

01. Introduction

➤ Limitations of Existing BPAD Approaches

- **Point-wise Event Modeling**
 - Most DL-based methods treat each event as an individual token and do not explicitly model local subsequences.
 - An event may appear normal in isolation, while its surrounding subsequence deviates from the expected process flow [24].
- **Long Trace Modeling**
 - Modeling long-range dependencies becomes increasingly expensive as trace length grows [25].
 - Self-attention scales quadratically with sequence length [26].

→ BPAD requires an efficient representation of local process patterns in long event logs.

PaCHITA addresses both limitations through overlapping patches, capturing local sequential dependencies while reducing the effective sequence length.

01. Introduction

➤ Contributions

- PaCHITA: A **P**atched **C**hannel-**I**ndependent **T**ransformer for Business Process **A**nomaly Detection
- **Overlapping Patch Representation**
 - Segments each trace into overlapping patches of consecutive events.
 - Captures local co-occurrence patterns while reducing the effective sequence length.
- **Channel-Independent Encoding**
 - Processes the activity and each attribute through separate encoder channels.
 - Preserves attribute-specific information throughout the model.
- **Unsupervised Fine-grained Detection**
 - Learns from event logs without requiring a clean log or prior process knowledge.
 - Detects anomalous traces and identifies the attribute values responsible for the anomaly.

02. Background

➤ Event Log Representation in PaCHITA

- **Patch-based Representation**

Event sequence: $\langle A, B, C, D, E, F \rangle$

Overlapping patches ($w = 3, st = 1$):

$\langle A, B, C \rangle \rightarrow \langle B, C, D \rangle \rightarrow \langle C, D, E \rangle \rightarrow \langle D, E, F \rangle$

- Consecutive events are grouped into overlapping patches representing local process patterns.
- Patching captures local co-occurrence while reducing the sequence length processed by self-attention.

- **Channel-Independent Modeling**

Activity: $\langle A, B, C \rangle, \langle B, C, D \rangle, \langle C, D, E \rangle, \langle D, E, F \rangle$

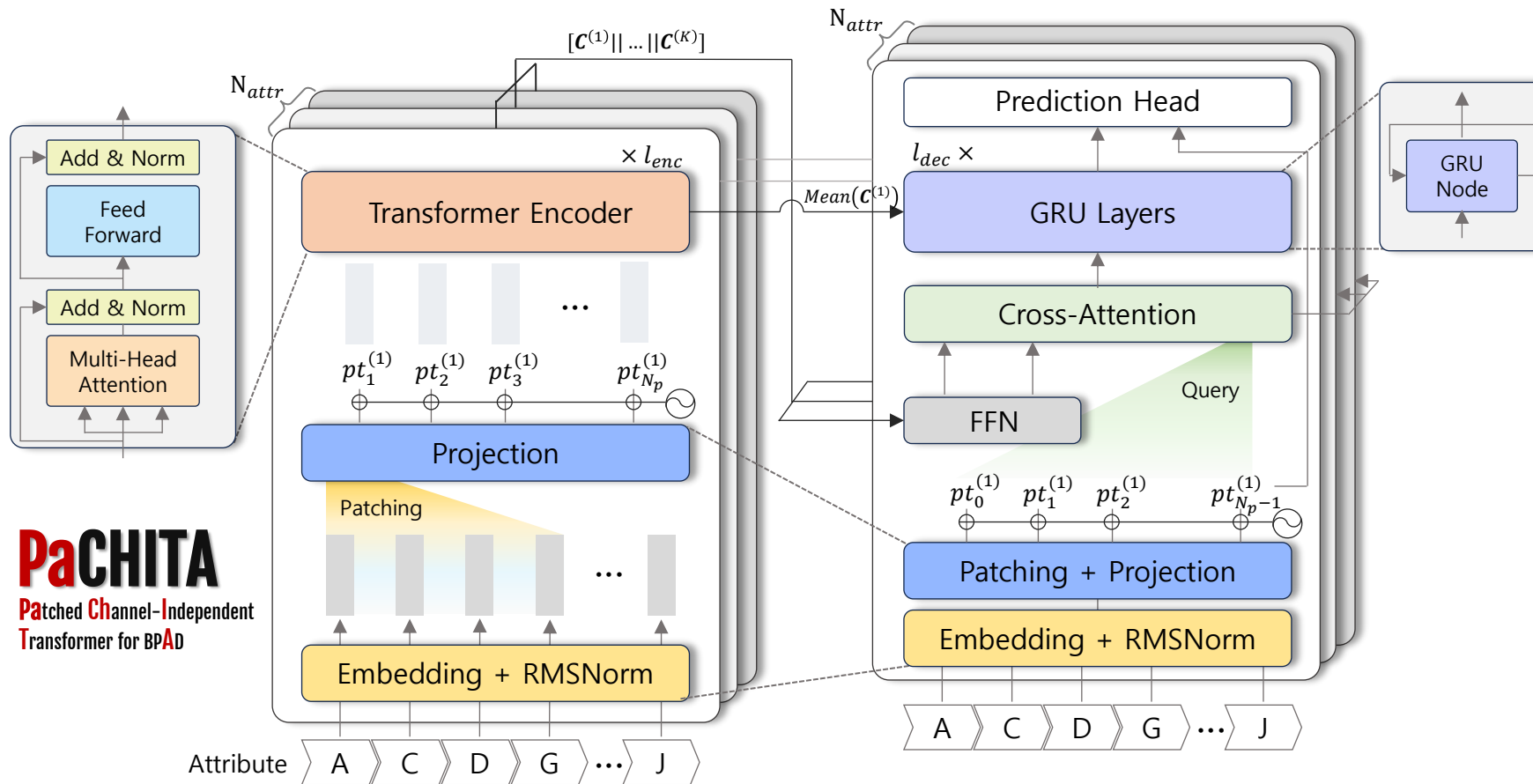
Resource: $\langle R_1, R_2, R_3 \rangle, \langle R_2, R_3, R_4 \rangle, \langle R_3, R_4, R_5 \rangle, \langle R_4, R_5, R_6 \rangle$

Department: $\langle D_1, D_2, D_3 \rangle, \langle D_2, D_3, D_4 \rangle, \langle D_3, D_4, D_5 \rangle, \langle D_4, D_5, D_6 \rangle$

- Activity and individual attributes are represented as separate, position-aligned channels.
- Each channel is encoded independently to preserve channel-specific patterns before cross-channel fusion.

03. Proposed Method

➤ Overview of the Proposed PaCHITA Framework



03. Proposed Method

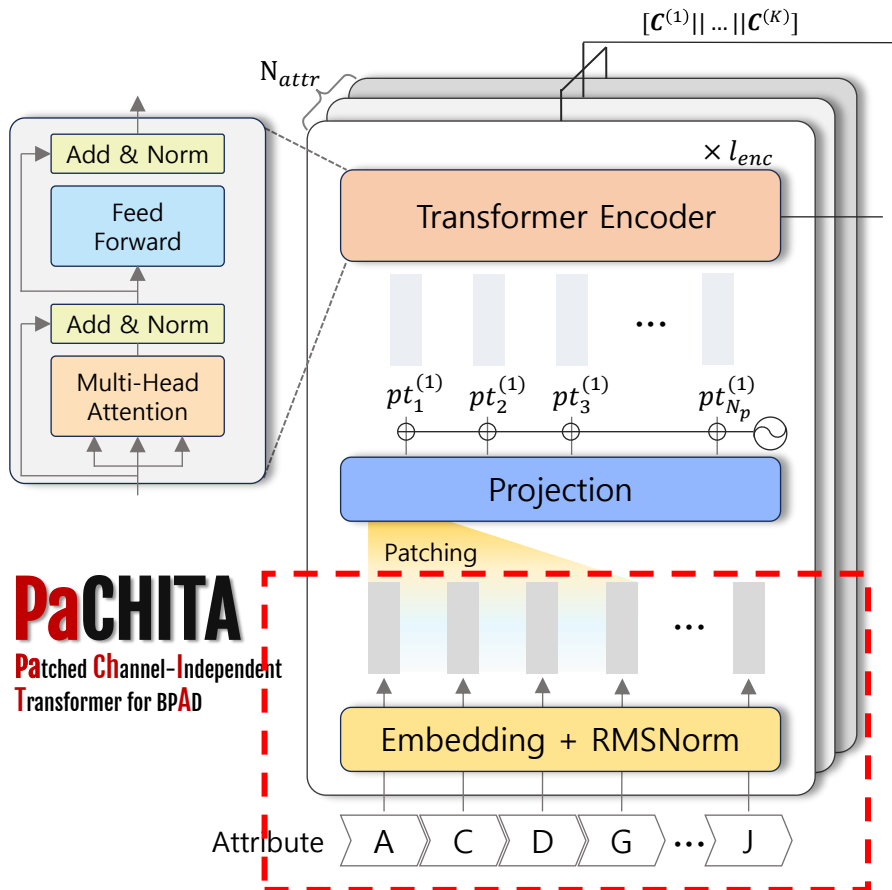
➤ Channel-Independent Patch Encoding

• Embedded Channel Sequence

$$\mathbf{x}^{(k)} = \left(x_1^{(k)}, \dots, x_T^{(k)} \right), \quad x_t^{(k)} \in \mathbb{R}^{d_{\text{emb}}}$$

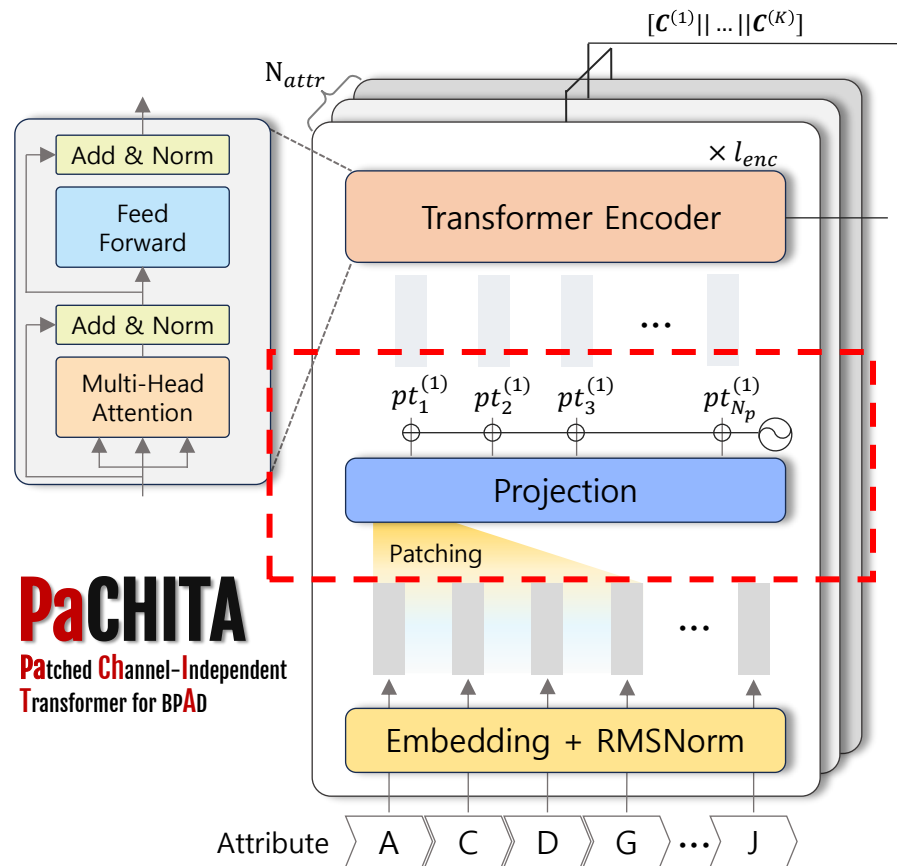
- Categorical values in each channel k are mapped to d_{emb} -dimensional token embeddings.
- $\mathbf{x}^{(k)}$ denotes the resulting channel sequence; shorter traces are right-padded to the maximum trace length T .

k : channel index, t : event position, T : maximum trace length



03. Proposed Method

➤ Channel-Independent Patch Encoding



• Overlapping Patch Construction

$$\left\{ x_t^{(k)}, x_{t+1}^{(k)}, \dots, x_{t+w-1}^{(k)} \right\}_{t=1}^{N_p}$$

- Each patch contains w consecutive embedded tokens.
- With $st = 1$, adjacent patches shift by one position and share $w - 1$ tokens.

• Patch Embedding Projection

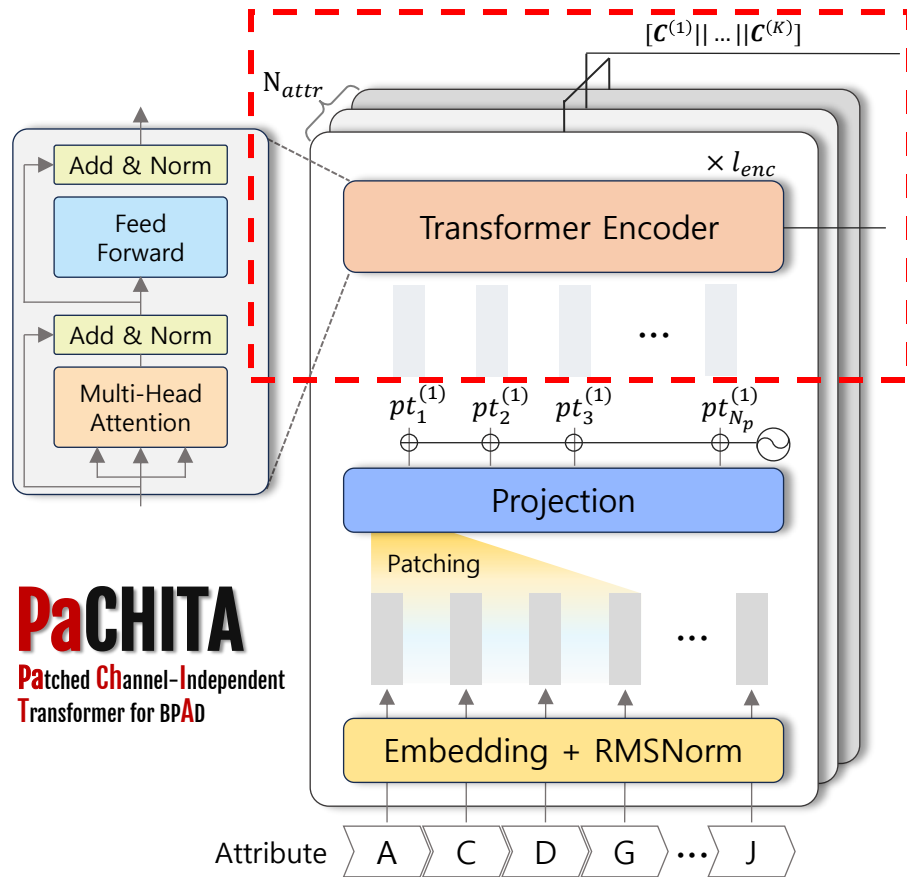
$$pt_t^{(k)} = \mathbf{W}_{\text{proj}} \left[\text{RMSNorm} \left(x_t^{(k)} \right) \parallel \dots \parallel \text{RMSNorm} \left(x_{t+w-1}^{(k)} \right) \right] \in \mathbb{R}^{d_{\text{model}}}$$

- The normalized token embeddings are concatenated and projected into a fixed d_{model} -dimensional patch representation.
- A learnable positional embedding is added to preserve the patch order.

t : patch start position, j : position within a patch, N_p : number of patches

03. Proposed Method

➤ Channel-Independent Patch Encoding



• Channel-Independent Transformer Encoding

$$\mathbf{C}^{(k)} = \text{Trn}_{\text{enc}}^{(k)}(\mathbf{pt}^{(k)}) \in \mathbb{R}^{N_p \times d_{\text{model}}}$$

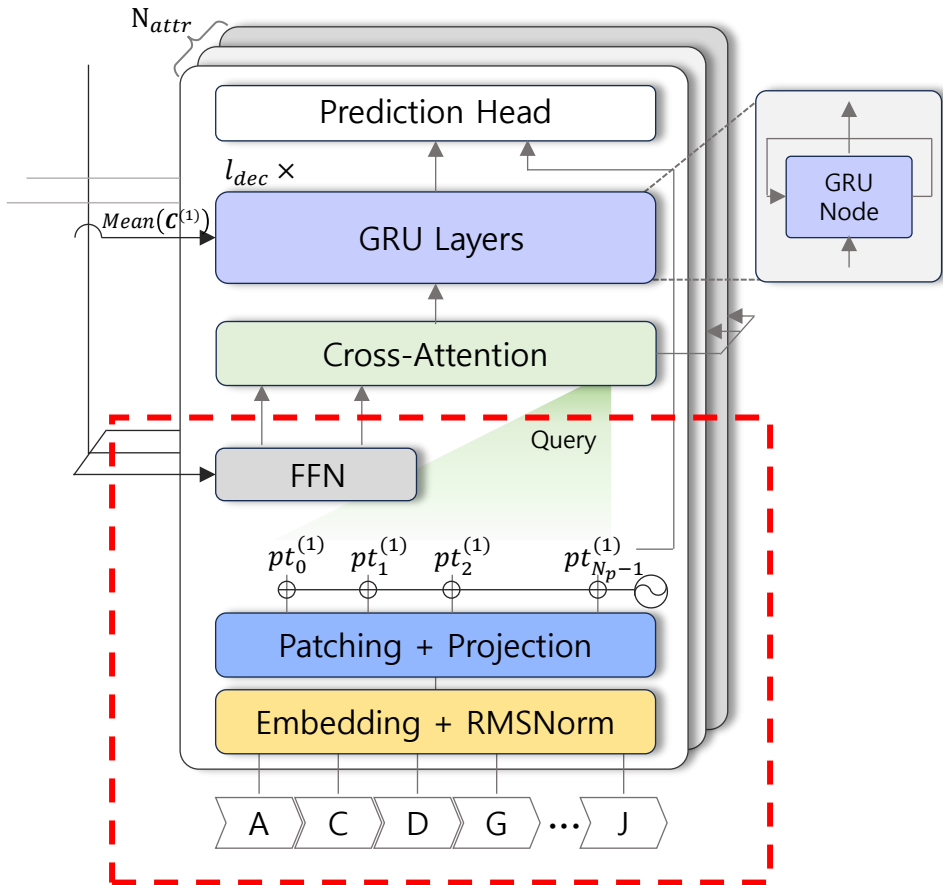
- $\mathbf{pt}^{(k)} = \langle pt_1^{(k)}, \dots, pt_{N_p}^{(k)} \rangle$ denotes the patch-embedding sequence of channel k .
- Each channel is processed by a dedicated Transformer encoder; encoder parameters are not shared across channels.
- $\mathbf{C}^{(k)}$ is the encoded patch sequence, retaining channel-specific patterns before cross-channel fusion.

$k = 1$: activity channel, $k = 2, \dots, K$: attribute channels

$K = 1 + N_{\text{attr}}$: number of channels

03. Proposed Method

➤ Cross-Channel Fusion and Shifted Decoder Input



• Cross-Channel Fusion

$$\mathbf{F}^{(k)} = \text{FFN}^{(k)}([\mathbf{C}^{(1)} \parallel \dots \parallel \mathbf{C}^{(K)}]) \in \mathbb{R}^{N_p \times d_{\text{model}}}$$

- At each patch position, encoder outputs from all K channels are concatenated.
- A decoder-specific $\text{FFN}^{(k)}$ transforms the combined information into $\mathbf{F}^{(k)}$, allowing each decoder to use all channels while adapting the representation to its target channel.

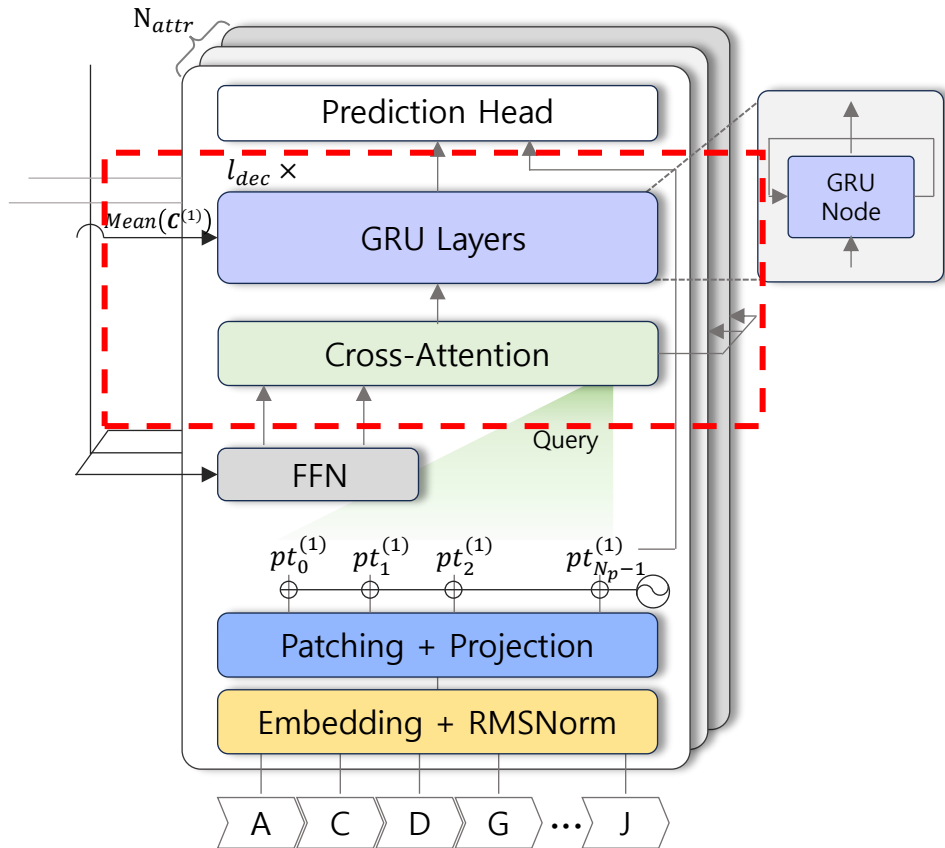
• Shifted Decoder Input

$$pt_{dec,t}^{(k)} = \begin{cases} [0, \dots, 0, x_1^{(k)}], & t = 1, \\ pt_{t-1}^{(k)}, & t \geq 2 \end{cases}$$

- During teacher forcing, the previous observed patch is used as the decoder input.
- First step: Input $\langle 0, 0, A \rangle \rightarrow$ Target $\langle A, B, C \rangle$.
- Next step: Input $\langle A, B, C \rangle \rightarrow$ Target $\langle B, C, D \rangle$.

03. Proposed Method

➤ Cross-Attention and GRU-based Decoding



• Cross-Attention

$$\mathbf{h}_{act} = \text{RMSNorm} \left(\text{Attn} \left(\mathbf{pt}_{dec}^{(1)}, \mathbf{F}^{(1)}, \mathbf{F}^{(1)} \right) + \mathbf{pt}_{dec}^{(1)} \right)$$

$$\mathbf{h}_{attr}^{(k)} = \text{RMSNorm} \left(\text{Attn} \left(\mathbf{pt}_{dec}^{(k)}, \mathbf{F}^{(k)}, \mathbf{F}^{(k)} \right) + \mathbf{pt}_{dec}^{(k)} \right) \quad k = 2, \dots, K$$

- The shifted decoder input serves as the query, while $\mathbf{F}^{(k)}$ provides the key and value.
- The resulting context is passed to its corresponding GRU.

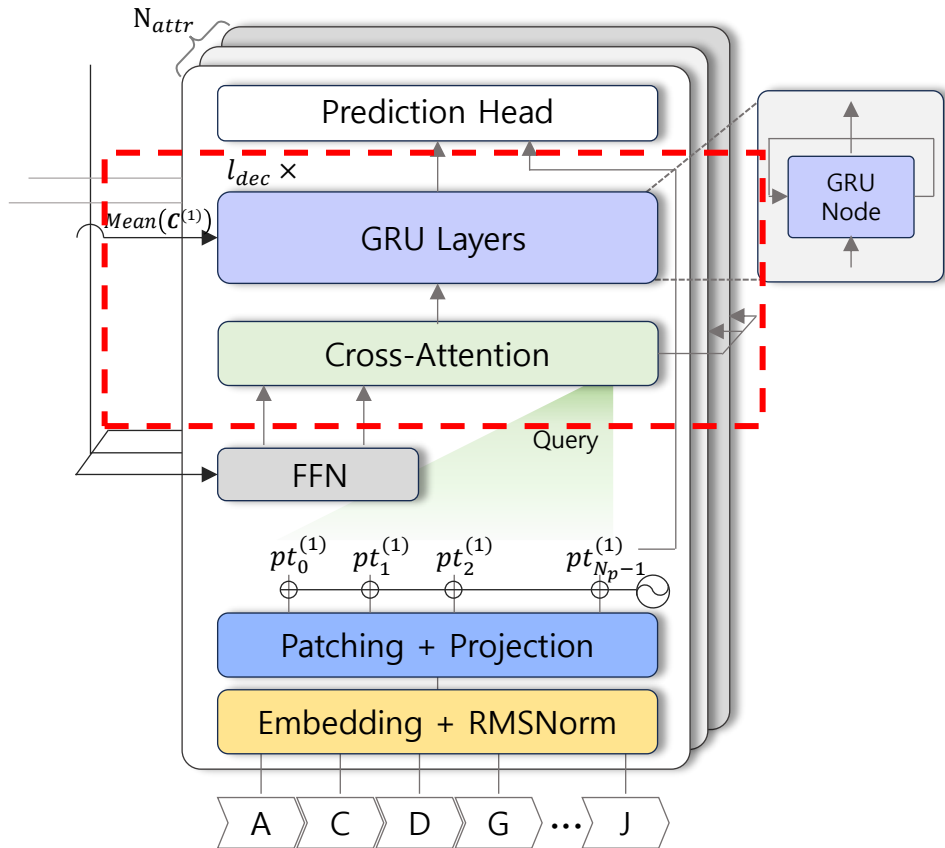
• GRU-based Activity Decoding

$$\mathbf{g}_{act} = \text{GRU} \left(\mathbf{h}_{act}, h_0 = \text{mean}(\mathbf{C}^{(1)}) \right)$$

- The activity GRU processes $\mathbf{h}_{act}$ across patch steps to capture sequential activity dependencies.
- Its initial hidden state h_0 is obtained by mean-pooling $\mathbf{C}^{(1)}$ over valid patch positions.

03. Proposed Method

➤ Cross-Attention and GRU-based Decoding



• Cross-Attention

$$\mathbf{h}_{act} = \text{RMSNorm} \left(\text{Attn} \left(\mathbf{pt}_{dec}^{(1)}, \mathbf{F}^{(1)}, \mathbf{F}^{(1)} \right) + \mathbf{pt}_{dec}^{(1)} \right)$$

$$\mathbf{h}_{attr}^{(k)} = \text{RMSNorm} \left(\text{Attn} \left(\mathbf{pt}_{dec}^{(k)}, \mathbf{F}^{(k)}, \mathbf{F}^{(k)} \right) + \mathbf{pt}_{dec}^{(k)} \right) \quad k = 2, \dots, K$$

- The shifted decoder input serves as the query, while $\mathbf{F}^{(k)}$ provides the key and value.
- The resulting context is passed to its corresponding GRU.

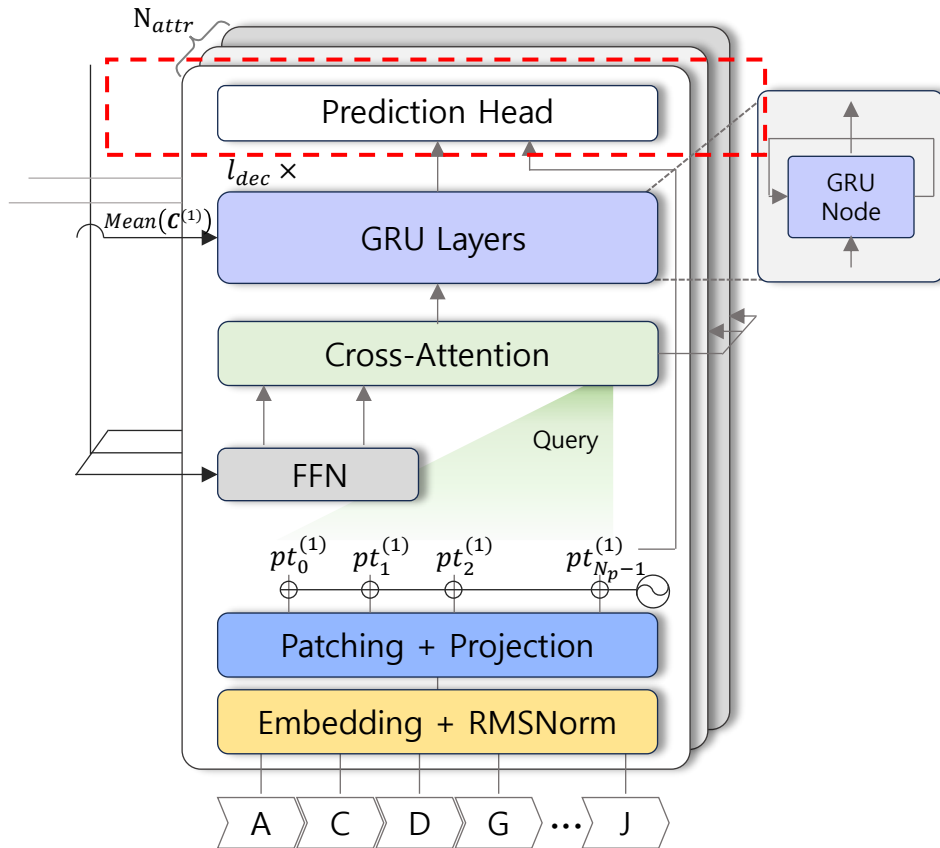
• Activity-Conditioned Attribute Decoding

$$\mathbf{g}_{attr}^{(k)} = \text{GRU}^{(k)} \left(\left[\mathbf{h}_{attr}^{(k)} \parallel \mathbf{h}'_{act} \right], h_0 = \text{mean}(\mathbf{C}^{(k)}) \right)$$

- The attribute GRU combines its channel context $\mathbf{h}_{attr}^{(k)}$ with the detached activity context $\mathbf{h}'_{act}$.
- Activity conditioning provides control-flow information for predicting attribute values within the corresponding process step.
- Detachment makes the conditioning one-way, preventing attribute losses from updating the activity decoder.

03. Proposed Method

➤ Channel-Specific Prediction Heads



• Activity Prediction

$$\hat{\mathbf{Y}}^{(1)} = \mathbf{W}_{\text{head}}^{(1)} \left[\mathbf{pt}_{\text{dec}}^{(1)} \parallel \mathbf{g}_{\text{act}} \right] \in \mathbb{R}^{N_p \times w \times (V_1 + 1)}$$

- The activity head combines the shifted decoder input with the activity GRU output.

• Attribute Prediction

$$\hat{\mathbf{Y}}^{(k)} = \mathbf{W}_{\text{head}}^{(k)} \left[\mathbf{pt}_{\text{dec}}^{(k)} \parallel \mathbf{h}'_{\text{act}} \parallel \mathbf{g}_{\text{attr}}^{(k)} \right] \in \mathbb{R}^{N_p \times w \times (V_k + 1)}$$

- Each attribute head combines its shifted decoder input, the detached activity context $\mathbf{h}'_{\text{act}}$, and its attribute GRU output $\mathbf{g}_{\text{attr}}^{(k)}$.
- For each within-patch position, each head produces a logit vector over all possible values in its channel and a padding token. Softmax converts the logits into a probability distribution.

V_k : number of possible values in channel k , +1 : padding token

03. Proposed Method

➤ Experimental Training and Evaluation Flow

1. Data Preparation

- Original Event Log → Synthetic Anomaly Injection → Anomaly-Injected Event Log
 - Anomaly labels generated during injection are stored separately.

2. Label-Free Training

- Anomaly-Injected Event Log → CE-based Training → Trained PaCHITA

3. Detection and Evaluation

- Trained PaCHITA → Anomaly Scoring → F1-score Evaluation
 - Stored anomaly labels are used only for evaluation.

• Training Objective

$$\mathcal{L} = \sum_{k=1}^K \mathcal{L}_{\text{CE}}^{(k)}, \quad \mathcal{L}_{\text{CE}}^{(k)} = - \sum_{t,j} \log p \left(x_{t+j}^{(k)} \mid \hat{\mathbf{Y}}_{t,j}^{(k)} \right)$$

- Observed tokens, including those at anomalous positions, are used as CE targets; anomaly labels are not provided to the model.
- The loss is summed over all non-padding positions and K channels.

03. Proposed Method

➤ Anomaly Score Measurement

- Per-position Score

$$\text{idx}_{n_p,j}^{(k)} = \pi \left(x_{n_p,j}^{(k)} \right), \quad s_{n_p,j}^{(k)} = \sum_{v: p(v) > p \left[\text{idx}_{n_p,j}^{(k)} \right]} p(v)$$

- Softmax converts the logits into a probability distribution $p(v)$, while $\text{idx}_{n_p,j}^{(k)}$ identifies the observed token.
- The score sums the probability mass of candidate values ranked above the observed token.
- This captures all stronger alternatives, rather than only the most likely one, while excluding candidates ranked below the observation.
- The score is 0 when the observed token is top-ranked; a higher score indicates stronger inconsistency with the learned context.

n_p : output patch index, j : position within the output patch

k : channel index, idx : observed token index

03. Proposed Method

➤ Anomaly Score Aggregation and Evaluation

- **Overlap Aggregation**

$$s_i^{(k)} = \max_{\{(n,j): n+j=i\}} s_{n,j}^{(k)}$$

- For each event position and channel, the maximum score across overlapping patches is retained as the attribute-level score.

- **Trace-level Score**

$$s_\sigma = \max_{i,k} s_i^{(k)}$$

- The maximum score across all event positions and channels becomes the trace-level score.

- **Detection and Evaluation**

$$\hat{y} = 1 \text{ if } s \geq \tau; \text{ otherwise } 0$$

(1 : anomalous, 0 : normal)

- A score at or above τ is classified as anomalous.
- For each method, τ is varied over its anomaly-score values, and the maximum F1-score is reported.

04. Experiments

➤ Experimental Settings

- **Artificial Anomalies** [19]

- 1) Skip: A sequence of up to two events is skipped.
- 2) Insert: Up to two random activities are inserted at arbitrary positions within a trace.
- 3) Rework: A sequence of up to three activities is repeated.
- 4) Early: A sequence of up to two activities moves to an earlier position in a trace.
- 5) Late: A sequence of up to two activities moves to a later position in a trace.
- 6) Attribute: Some values of attributes are mutated into incorrect values within a trace.

- **Event Logs**

Table 1. Statistical Descriptions of the Real-life Event Logs

Event Log	#Cases	#Events	Max length	Min length	#Attributes	#Activities
Domestic	10,500	56,437	24	1	2	17
International	6,449	72,151	27	3	2	34
Prepaid	2,099	18,246	21	1	2	29
Request	6,886	36,796	20	1	2	19
Receipt	1,434	8,577	25	1	2	27

- **Parameter Settings**

- Model Configuration. $w = 3$, $st = 1$; 2-layer Transformer (4 heads) and 2-layer GRU; hidden size 64.
- Training. 16 epochs, $lr = 3 \times 10^{-4}$; batch size 16 for logs with $\leq 5,000$ cases, otherwise 64.

04. Experiments

➤ Experimental Results: Performance Evaluation

- Anomalies were injected into 10% of the traces, while no anomaly labels were used for training.

Table 2. Performance Comparison (F1-score) for both Trace-level and Attribute-level over five real-life event logs

Models \ Logs	Domestic		International		Prepaid		Request		Receipt	
	Trace	Attr	Trace	Attr	Trace	Attr	Trace	Attr	Trace	Attr
Sampling	0.611	-	0.351	-	0.514	-	0.675	-	0.489	-
Naïve	0.767	-	0.627	-	0.647	-	0.742	-	0.593	-
GAE	0.415	-	0.290	-	0.314	-	0.438	-	0.278	-
VAE-OCSVM	0.174	-	0.190	-	0.186	-	0.191	-	0.183	-
W2V-LOF	0.636	-	0.491	-	0.519	-	0.681	-	0.594	-
LAE	0.812	0.502	0.597	0.440	0.527	0.453	0.780	0.439	0.396	0.299
VAE	0.502	0.168	0.295	0.127	0.305	0.124	0.484	0.141	0.436	0.100
BINet	0.507	0.499	0.570	0.468	0.564	0.471	0.638	0.531	0.609	0.400
GRASPED	0.821	0.466	0.746	0.540	0.602	0.429	0.806	0.482	0.469	0.418
PaCHITA	0.835	0.512	0.759	0.554	0.762	0.516	0.829	0.511	0.622	0.477

• Trace-level performance

- PaCHITA achieved the highest trace-level F1-scores across all five event logs.
- It outperformed the strongest baseline, GRASPED, on the Domestic (**0.835 vs. 0.821**) and Request logs (**0.829 vs. 0.806**).
- The largest gain was observed on Prepaid, outperforming Naïve by 11.5 percentage points (**0.762 vs. 0.647**); it also surpassed BINet on Receipt (**0.622 vs. 0.609**).

04. Experiments

➤ Experimental Results: Performance Evaluation

- Anomalies were injected into 10% of the traces, while no anomaly labels were used for training.

Table 2. Performance Comparison (F1-score) for both Trace-level and Attribute-level over five real-life event logs

Models \ Logs	Domestic		International		Prepaid		Request		Receipt	
	Trace	Attr	Trace	Attr	Trace	Attr	Trace	Attr	Trace	Attr
Sampling	0.611	-	0.351	-	0.514	-	0.675	-	0.489	-
Naïve	0.767	-	0.627	-	0.647	-	0.742	-	0.593	-
GAE	0.415	-	0.290	-	0.314	-	0.438	-	0.278	-
VAE-OCSVM	0.174	-	0.190	-	0.186	-	0.191	-	0.183	-
W2V-LOF	0.636	-	0.491	-	0.519	-	0.681	-	0.594	-
LAE	0.812	0.502	0.597	0.440	0.527	0.453	0.780	0.439	0.396	0.299
VAE	0.502	0.168	0.295	0.127	0.305	0.124	0.484	0.141	0.436	0.100
BINet	0.507	0.499	0.570	0.468	0.564	0.471	0.638	0.531	0.609	0.400
GRASPED	0.821	0.466	0.746	0.540	0.602	0.429	0.806	0.482	0.469	0.418
PaCHITA	0.835	0.512	0.759	0.554	0.762	0.516	0.829	0.511	0.622	0.477

• Attribute-level performance

- PaCHITA achieved the highest attribute-level F1-scores on four logs and the second-highest score on Request.
- The largest gains were observed on Prepaid and Receipt, outperforming BINet and GRASPED by **4.5** and **5.9 percentage points**, respectively.
- VAE remained below **0.17** across all logs, suggesting that its reconstruction-based approach struggles with fine-grained anomaly localization.

04. Experiments

➤ Experimental Results: Training Efficiency

Table 3. Training time comparison (in seconds) across all methods

	Sampling	Naïve	GAE	VAE-OCSVM	W2V-LOF	LAE	VAE	BiNet	GRASPED	PaCHITA
Time (sec)	1.4	1.2	145.9	29.7	2.8	89.9	95.9	113.3	301.3	117.6

- **Statistical and Non-parametric Methods**

- Naïve (**1.2 s**) and Sampling (**1.4 s**) were the fastest, as they rely on statistical operations without parameterized models.
- W2V-LOF (**2.8 s**) uses pre-trained word embeddings with a non-parametric outlier detection method.

- **Channel-independent Neural Models**

- GRASPED, which also uses a channel-independent architecture with self-attention, required the longest training time (**301.3 s**).
- PaCHITA completed training in **117.6 s**, approximately **2.6 × faster** than GRASPED.
- This efficiency gain is attributed to patching, which reduces the effective sequence length processed by self-attention.

04. Experiments

➤ Experimental Results: Sensitivity to the Anomaly Ratio

- Anomalies were injected at ratios ranging from 5% to 40% across five real-life event logs.
- Trace-level F1-scores were evaluated under varying anomaly ratios.

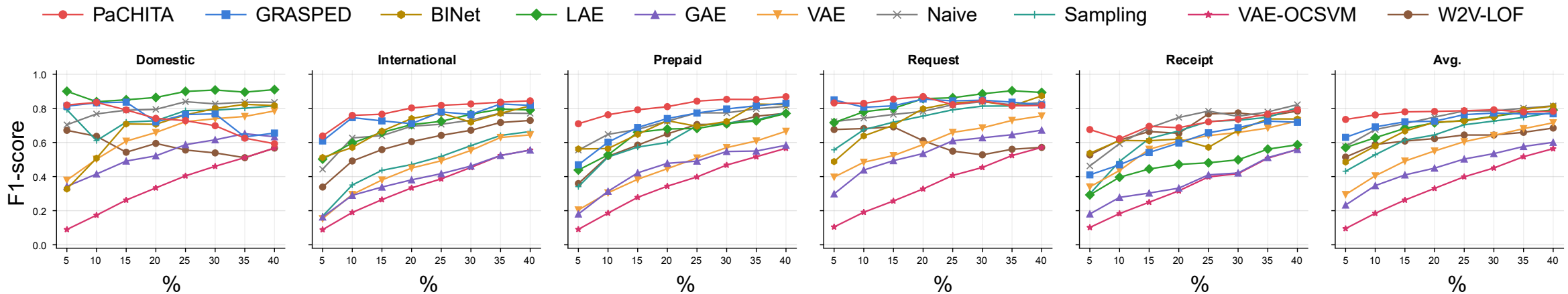


Figure 3(a). Trace-level Performance across Varying Anomaly Ratios

• Trace-level Performance

- PaCHITA achieved the highest or near-highest trace-level F1-scores across most event logs and anomaly ratios.
- At a 5% anomaly ratio, PaCHITA achieved an average F1-score of 0.735, outperforming GRASPED (0.630) by 10.5 percentage points.
- Its advantage was maintained across most anomaly ratios on Prepaid and Receipt, demonstrating the effectiveness of patching in capturing local sequential dependencies.
- On Domestic, the performance gain diminished at higher ratios because frequent anomalous patterns could be absorbed as normal behavior, combined with low activity-pattern variability.

04. Experiments

➤ Experimental Results: Sensitivity to the Anomaly Ratio

- Anomalies were injected at ratios ranging from 5% to 40% across five real-life event logs.
- Attribute-level F1-scores were evaluated under varying anomaly ratios.

● PaCHITA
 ■ GRASPED
 ◆ BINet
 ◇ LAE
 ▲ GAE
 ▼ VAE
 × Naive
 + Sampling
 ★ VAE-OCSVM
 ● W2V-LOF

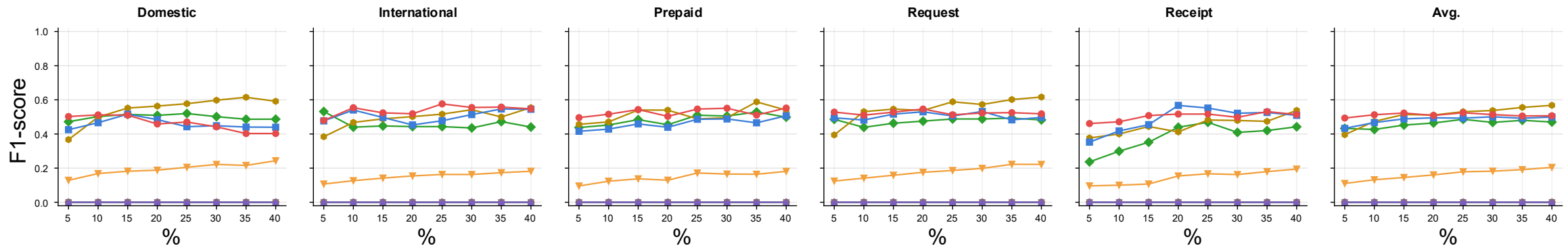


Figure 3(b). Attribute-level Performance across Varying Anomaly Ratios

• Attribute-level Performance

- PaCHITA achieved strong attribute-level detection performance across all event logs and anomaly ratios.
- Its average F1-score ranged from **0.493 (at 5%)** to **0.524 (at 25%)**, consistently surpassing GRASPED (**0.433–0.501**).
- LAE exhibited competitive but lower scores (**0.427–0.486**), while VAE remained below 0.21 across all ratios.
- Overall, PaCHITA was robust to varying anomaly ratios at both trace and attribute levels across diverse event logs.

05. Conclusion

➤ Conclusion

- **Research summary**

- PaCHITA outperformed nine baselines at both trace and attribute levels across five real-life event logs.
- Patching captures local dependencies while reducing the effective sequence length and computational cost.
- The channel-independent design preserves attribute-specific patterns for fine-grained anomaly detection.
- PaCHITA retains a clear advantage in the practical low-anomaly regime.

- **Limitations**

- At high anomaly ratios, frequent anomalies may be absorbed as normal behavior, reducing performance gains as observed on Domestic, which has limited variant diversity and short traces.
- Model size increases with the number of attributes because each attribute requires a dedicated channel stream.

- **Future Work**

- Combining patch-based local dependency capture with graph-based relational modeling to jointly represent:
 - ✓ Local co-occurrence patterns within patches.
 - ✓ Inter-event relational structures across the trace.

Thank you

ACKNOWLEDGEMENT

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2023-00218913).

References

- [1] M. Dumas, M. La Rosa, J. Mendling, H. A. Reijers, et al., *Fundamentals of Business Process Management*, Vol. 1, Springer, 2013.
- [2] W. van der Aalst, T. Weijters, and L. Maruster, "Workflow Mining: Discovering Process Models from Event Logs," *IEEE Transactions on Knowledge and Data Engineering*, Vol. 16, pp. 1128–1142, 2004.
- [3] S. Sim, H. Bae, and L. Liu, "Bagging Recurrent Event Imputation for Repair of Imperfect Event Log with Missing Categorical Events," *IEEE Transactions on Services Computing*, Vol. 16, pp. 108–121, 2021.
- [4] R. J. C. Bose, R. S. Mans, and W. M. van der Aalst, "Wanna Improve Process Mining Results?" 2013 IEEE Symposium on Computational Intelligence and Data Mining (CIDM), pp. 127–134, 2013.
- [5] W. M. van der Aalst and A. K. A. de Medeiros, "Process Mining and Security: Detecting Anomalous Process Executions and Checking Process Conformance," *Electronic Notes in Theoretical Computer Science*, Vol. 121, 2005.
- [6] F. Bezerra, J. Wainer, and W. M. van der Aalst, "Anomaly Detection Using Process Mining," *International Workshop on Business Process Modeling, Development and Support*, pp. 149–161, Springer, 2009.
- [7] D. Rahmawati, M. A. Yaqin, and R. Sarno, "Fraud Detection on Event Logs of Goods and Services Procurement Business Process Using Heuristics Miner Algorithm," *International Conference on Information & Communication Technology and Systems (ICTS)*, pp. 249–254, 2016.
- [8] F. Bezerra and J. Wainer, "Algorithms for Anomaly Detection of Traces in Logs of Process Aware Information Systems," *Information Systems*, Vol. 38, pp. 33–44, 2013.
- [9] W. van der Aalst, "Data Science in Action," in *Process Mining: Data Science in Action*, pp. 3–23, Springer, 2016.
- [10] W. van der Aalst, A. Adriansyah, and B. van Dongen, "Replaying History on Process Models for Conformance Checking and Performance Analysis," *WIREs Data Mining and Knowledge Discovery*, Vol. 2, pp. 182–192, 2012.
- [11] J. Ko and M. Comuzzi, "Keeping Our Rivers Clean: Information-Theoretic Online Anomaly Detection for Streaming Business Process Events," *Information Systems*, Vol. 104, Article 101894, 2022.
- [12] J. Ko and M. Comuzzi, "Detecting Anomalies in Business Process Event Logs Using Statistical Leverage," *Information Sciences*, Vol. 549, pp. 53–67, 2021.
- [13] S. Pauwels and T. Calders, "An Anomaly Detection Technique for Business Processes Based on Extended Dynamic Bayesian Networks," *ACM Symposium on Applied Computing*, pp. 494–501, 2019.
- [14] L. Yang, S. McClean, M. Donnelly, K. Khan, and K. Burke, "Analysing Business Process Anomalies Using Discrete-Time Markov Chains," *HPCC/SmartCity/DSS*, pp. 1258–1265, 2020.
- [15] D. Rahmawati, R. Sarno, C. Fatichah, and D. Sunaryono, "Fraud Detection on Event Log of Bank Financial Credit Business Process Using Hidden Markov Model Algorithm," *International Conference on Science in Information Technology (ICSITech)*, pp. 35–40, 2017.
- [16] T. Nolle, A. Seeliger, and M. Mühlhäuser, "BINet: Multivariate Business Process Anomaly Detection Using Deep Learning," *International Conference on Business Process Management*, pp. 271–287, Springer, 2018.
- [17] H. T. C. Nguyen, S. Lee, J. Kim, J. Ko, and M. Comuzzi, "Autoencoders for Improving Quality of Process Event Logs," *Expert Systems with Applications*, Vol. 131, pp. 132–147, 2019.
- [18] S. Huo, H. Völzer, P. Reddy, P. Agarwal, V. Isahagian, and V. Muthusamy, "Graph Autoencoders for Business Process Anomaly Detection," *International Conference on Business Process Management*, pp. 417–433, Springer, 2021.
- [19] T. Nolle, S. Luetzgen, A. Seeliger, and M. Mühlhäuser, "BINet: Multi-perspective Business Process Anomaly Classification," *Information Systems*, Vol. 103, Article 101458, 2022.
- [20] W. Guan, J. Cao, Y. Gu, and S. Qian, "GRASPED: A GRU-AE Network Based Multi-Perspective Business Process Anomaly Detection Model," *IEEE Transactions on Services Computing*, Vol. 16, pp. 3412–3424, 2023.
- [21] T. Nolle, A. Seeliger, and M. Mühlhäuser, "Unsupervised Anomaly Detection in Noisy Business Process Event Logs Using Denoising Autoencoders," *International Conference on Discovery Science*, pp. 442–456, Springer, 2016.
- [22] W. Guan, J. Cao, Y. Gu, and S. Qian, "GAMA: A Multi-Graph-Based Anomaly Detection Framework for Business Processes via Graph Neural Networks," *Information Systems*, Vol. 124, Article 102405, 2024.
- [23] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, Vol. 521, pp. 436–444, 2015.
- [24] S. Sim, H. Bae, and Y. Choi, "Likelihood-Based Multiple Imputation by Event Chain Methodology for Repair of Imperfect Event Logs with Missing Data," 2019 International Conference on Process Mining (ICPM), pp. 9–16, 2019.
- [25] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series Is Worth 64 Words: Long-Term Forecasting with Transformers," *arXiv:2211.14730*, 2022.
- [26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, Vol. 30, 2017.
- [27] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," *Advances in Neural Information Processing Systems*, Vol. 32, 2019.